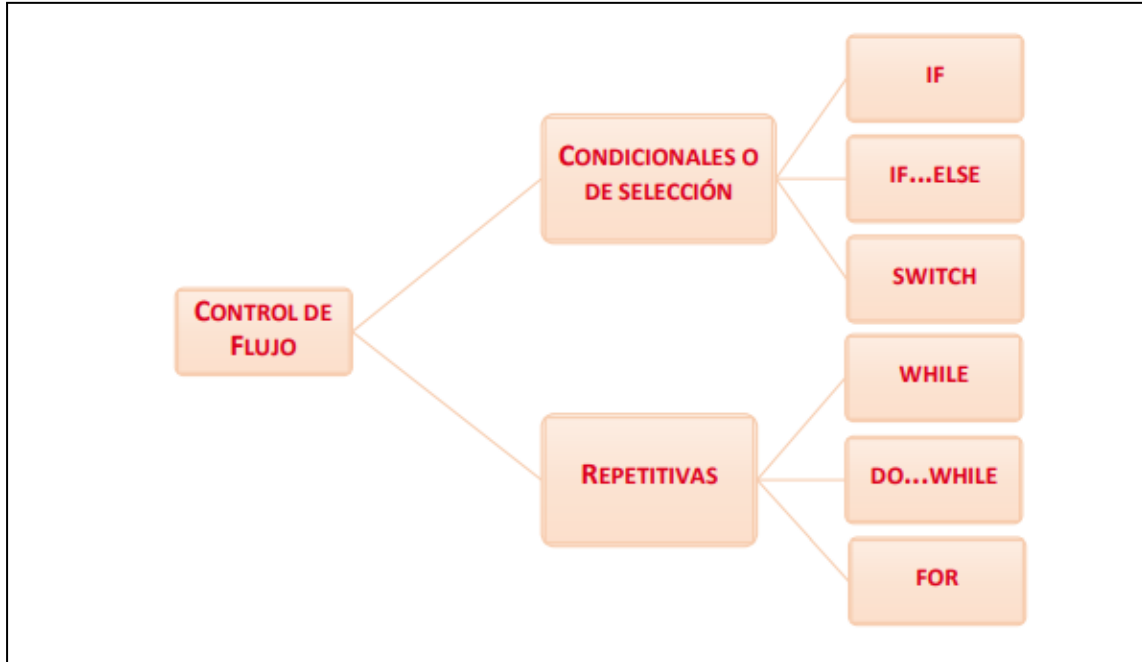


APUNTES TEMA 2

06/10/2025

SENTENCIAS DE CONTROL DE FLUJO



Las sentencias de control de flujo se encargan principalmente de contener otras sentencias del programa. Estas sentencias pueden indicar el orden de ejecución y bajo qué condiciones.

CONDICIONALES DE SELECCIÓN

IF: se utiliza para las condiciones cuando se cumplen, son bloques de códigos que se activan cuando la condición es true.

```
if (edad >= mayoría) {  
    System.out.println("Usted puede pasar");  
}  
System.out.println("Gracias por utilizar el programa.");
```

IF... ELSE: se utiliza para dar opciones cuando la condición se cumple o no, son bloques de código que se ejecutarán según si la condición es verdadera o falsa. Importante: **else** no pueden ir si no hay un **if**.

```
if (edad >= mayoría) {  
    System.out.println("Usted puede pasar");  
} else {  
    System.out.println("Eres menor, no puedes pasar.");  
}  
System.out.println("Gracias por utilizar el programa.");
```

-- Se pueden mas de una condición de forma anidadas, estas se llaman *if anidados* --

¿Qué significa poner condiciones múltiples? → Se utiliza para establecer varias condiciones en una misma línea de if, ambas variables deben ser booleanas, y no se suele usar más de tres condiciones en la misma línea, en su caso es recomendable usar un switch, su sintaxis hace que se puedan dividir entre varias formas:

- Se dividen entre (&&) y ambas deben ser booleanos true para poder llevarla a cabo la acción..

EJEMPLO: if (dinero>1,20 & & Cantidad !=0) {

- Se dividen entre (| |) y esto significa que una de las dos condiciones tiene que ser true para poder ejecutarse.

EJEMPLO: if (dinero>1,20 || Cantidad !=0) {

- - Estas dos variables se pueden anidar, pero hay que tener en cuenta la tabla de preferencias - -

SWITCH:

Los switch se utilizan para cuando hayan muchas elecciones de diferentes opciones. En la sintaxis de switch no se puede usar una condición sino que tiene que ser una expresión, esto hace referencia al nombre de una variable.

De qué tipo de dato pueden ser los switch:

- **enteros**

Recomendación: no utilizar switch para strings, char tampoco es recomendable (si se utilizan van entre comillas), ni para doubles.

sintaxis switch: se suele definir como “**case + valor que queremos que tenga + líneas de código + break;**”

- **case** se utiliza para determinar los casos posibles entre los valores propuestos.
- **default** se utiliza para todos aquellos valores que no están definidos en los casos.

- - **Al default es al unico que no le hace falta poner un break, para case es necesario usarlo sino se van a cargar los diferentes casos - -**

Ejemplo:

```
switch (planeta) {
    case 1:
        solucion= peso*constM;
        //submenu idiomas
        System.out.printf("\nElige la opción de idioma de su respuesta: \nPulse 1 para manterner en %s\nPulse 2 para traducir al %s",idom1,idom2);
        idioma=Leer.datoInt();
        switch (idioma) {
            case 1:
                System.out.printf("\nSu peso en %s sera de %.2f N",plant1,solucion);
                break;
            case 2:
                System.out.printf("\nYour weight on %s will be %.2f N",plant1, solucion);
                break;
            default:
                System.out.println("Opción de idioma incorrecto.");
        }
    }
}
```

Examen es un menu + do...while +...

hacer 5 y 8 para mañana

ESTRUCTURAS DE REPETICIÓN

En java encontramos diferentes sentencias iterativas (while, do...while, for)

WHILE: la estructura de while se lleva a cabo mediante **booleanos**, esto quiere decir que la condición debe ser **verdadera**. **IMPORTANTE** → evitar los bucles infinitos

Cómo funciona: En el bloque de while mientras la condición sea verdadera esta ejecutara el bloque del while de forma reiterada hasta que la condición sea **falsa**.

EJEMPLO SINTAXIS:

```
while (num1 < 7) {  
    num1=num1+1;  
    System.out.println("Se le ha sumado 1 unidad al valor introducido, ahora el valor es "+num1);  
}  
  
System.out.println("\n-----\nMuchas gracias su valor ya es apto.");  
}
```

```
Podrias introducir un valor  
4  
Se le ha sumado 1 unidad al valor introducido, ahora el valor es 5  
Se le ha sumado 1 unidad al valor introducido, ahora el valor es 6  
Se le ha sumado 1 unidad al valor introducido, ahora el valor es 7  
  
-----  
Muchas gracias su valor ya es apto.
```

- - En programación se cuenta empezando desde el cero, hay que diferenciar entre la forma de contar entre programador y usuario - -

- - En java para sumar valores a una variable podemos utilizar la siguiente sintaxis (num++) esta suma una unidad a la variable; y la expresión (num+=[valor]) te permite sumar el valor que quieras - -

CÓMO FUNCIONA EL DESIGNAR UN STRING (No básico): Cuando inicializamos un string (ejemplo: string nombre= "abc") se reserva un espacio en la memoria para el **string**, sin embargo al ser una cadena el **string** la variable guarda la información como un patrón de número y caracteres (ejemplo: nombre [1@3F] y noLeido [1@4f]) y aunque hayamos metido lo mismo el sistema no lo reconoce como lo mismo, por lo que no podemos igualarlos mediante ==.

Para solucionar este error y poder igualar se llevaría a cabo el uso de la siguiente sintaxis: "[nombre variable].equals([nombre otra variable])", esto se encargará de reconocer las cadenas de string.

TAREA: ejercicio 11,12,14

DO...WHILE: funciona al igual que el **while**, sin embargo a diferencia de este, **do...while** ejecuta el cuerpo del bucle mínimo una vez antes de evaluar el resultado de la expresión del booleano.

IMPORTANTE: Por sistema no vamos a preguntarle al usuario sobre repetir el programa sino que por defecto vamos a hacer que se repita.

FOR: Sirve para recorrer código, funciona al igual que un bucle, cuando tenemos claro el número de veces que debemos repetir. **Estructura bloque misma** sintaxis que while

ESTRUCTURA:

```
for (variable1 (i) ; variable2 ; variable3) {  
    código (ejemplo:syso);  
}
```

-- En el espacio de la variable1 vamos a llevar a cabo una inicialización de variable en el mismo espacio y esta se llamará (i) y nos va a indicar desde dónde empezamos- -

-- En el espacio de la variable2 vamos a llevar a cabo una expresión booleana al igual que while - -

-- En el espacio de la variable3 vamos a llevar a cabo una expresión matemática con el objetivo de saber de cuanto en cuanto va - -

EJEMPLO:

```
int profesor;  
System.out.println("Cuantas veces escribo la frase");  
profesor=Leer.datoInt();  
  
for (int i=0;i<profesor;i++) {  
    System.out.println((i+1)+" No llegar tarde");  
}
```

ARRAYS

El objetivo de los arrays es que a diferencia de las variables que solo te permiten guardar un valor por espacio, los **arrays** nos permitirán guardar varios en un mismo espacio de memoria.

Curiosidad: Los arrays nos permiten almacenar en cualquier dimensión (primera, segunda, cuarta, séptima...) sin embargo lo haremos en dos dimensiones.

Problemas de los arrays: cuando le doy en tamaño al array no lo puedo **modificar**, solo se puede guardar un tipo de dato en los arrays. (Esto significa que si usas enteros, deberás de crear otro para los dobles).

CÓMO FUNCIONAN LOS ARRAYS: a los espacios de los arrays decimos que están indexados (comienza desde el 0 al igual que los for). Estos índices nos ayudarán a movernos por el array (comprender que se cuenta en modo **programador**).

COMO SE DECLARAN LOS ARRAYS E INSTANCIAN:

Hay dos formas válidas:

- (tipo de dato) [] identificador
- (tipo de dato) identificador []

- - En los arrays cuando agregamos valor significa “instanciar”, sería lo mismo que inicializar - -

CÓMO ESTABLECEMOS TAMAÑO PARA LOS ARRAYS:

(variable)= new int [tamaño de espacios];

EJEMPLO:

```
int [] edades;
int tam;
System.out.println("Indicar cuantas edades va a guardar.");
//Instanciamos
tam=Leer.datoInt();
edades =new int [tam];
```

CÓMO GUARDAR VALORES EN LOS ARRAYS: para guardar los valores deberemos de indicar entre los corchetes, el valor que pongamos debe ser el espacio del índice.

IMPORTANTE: Cuando preguntemos al usuario debemos recordad que ellos van a responder contando como **usuarios** y no como **programador**.

```
edades[1]=19;|
```

¿Y SI QUIERO INICIALIZARLO?:

```
int [] edades = {1,2,3,4,5};
int tam;

System.out.println("que valor quieres leer?");
tam=Leer.datoInt();

System.out.println(edades[tam]);
```

CÓMO IMPRIMIMOS UN ARRAYS: debemos saber diferenciar a la hora de escribir la variable, si imprimimos directamente la variable del array nos dará como resultado una dirección de memoria (**EJ: 7@32**), para poder ver el valor del array deberemos indicarlo mediante los corchetes.

```
System.out.println(edades[1]);|
```

COMO LEER UN ARRAY ENTERO EN BUCLES: Para leer un array entero no podemos asignar un valor sino utilizar **nombreArray.length**
RECORDAD: se escribe el nombre del **array** no del **tamaño**

```
for (int i=0; i<valor.length;i++) {
```

- - Nunca utilizar la letra 'ñ' para declarar una variable, usar opciones como 'nn o ny' - -

- - La variable que introduciremos dentro de los corchetes del array deberá ser la misma que la del bucle que estemos utilizando - -

```
for (int i=0; i<valor.length;i++) {  
    System.out.println("Diga un numero");  
    valor[i]=Leer.datoInt();
```

GENERAR NÚMEROS ALEATORIO EN JAVA

Es importante aclarar que en java no existen los números aleatorios, pero podemos crearlos mediante diferentes métodos:

Math.random():

Esta sintaxis nos genera un número aleatorio entre 0.0 y 0.1, sin embargo este no se podrá para valores enteros. Para solucionarlo podemos utilizar la siguiente sintaxis: `int valorEntero = (int)Math.floor(Math.random()*(N-M+1)+M);`

- - La anterior fórmula se utiliza para conseguir un número entero entre M y N con M menor que N - -

(este método no lo vamos a utilizar)

- - Hay que diferenciar entre Random (R mayúscula), (esta es una clase de java) y random (r minúscula) no lleva semilla - -

Clase java.util.Random

Al utilizar esta forma los número aleatorios se van a crear realizando muchas operaciones matemáticas sobre un primer número que se denomina "Semilla".

Para que la semilla esté variando todo el tiempo utilizaremos un numero que pueda variar, para ello utilizaremos el tiempo (constante que siempre varía) y para ser más preciso utilizaremos los **nano segundos**.

SINTAXIS RANDOM:

Cuando introducimos la sintaxis **nos puede aparecer un error**, ya que no tenemos importada la clase **Random**

```
import java.util.Random;
```

Ejemplo Código:

```
int hasta, desde, ejemploAleatorio, veces;

System.out.println("Esto es un ejemplo de como crear aleatorios\n");
System.out.println("Introduzca el minimo valor que va a comprender el aleatorio:");
desde=Leer.datoInt();

System.out.println("\nIntroduzca el maximo valor que va a comprender el aleatorio: ")
hasta=Leer.datoInt();
Random r= new Random (System.nanoTime());

System.out.println("Cuantos aleatorios vas a querer crear: ");
veces=Leer.datoInt();

for (int i=0; i<veces; i++) {
    ejemploAleatorio=r.nextInt(hasta-desde+1)+desde;
    System.out.println("Ha salido el: "+ejemploAleatorio);
}
```

RECORDAD: Debemos tener un bucle para guardar valores y otro bucle independiente para mostrar los valores.

Cuando estemos en los arrays, cuando queramos utilizar el tamaño de los arrays tanto para bucles como para operaciones como medias NUNCA deberemos usar la variable del tamaño sino que usaremos la sintaxis (nombre array).length
